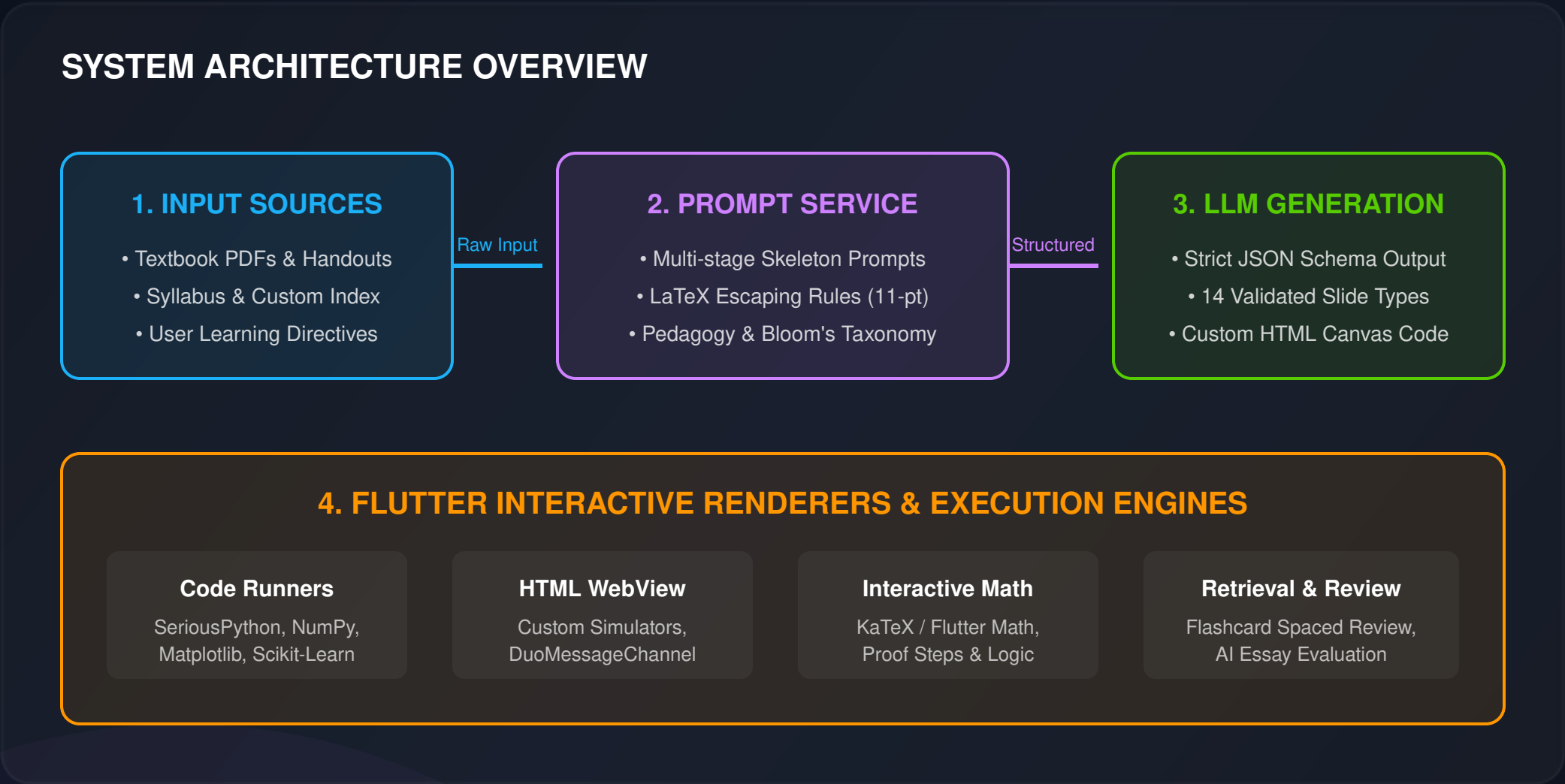


Duofy Architecture & Interactive Slide Types & AI Prompt Pipeline

A comprehensive documentary on the 14 interactive slide types, LLM prompt engineering architecture, LaTeX math formatting rules, and multi-stage course generation pipeline in Duofy.



DOCUMENTARY TABLE OF CONTENTS

Section 1: The 14 Interactive Slide Types in Duofy

- Theory, Concept Pieces, Quiz, Fill in Blank, One Word, Numerical, Proof/Step-by-Step
- Matching, Ordering, Flashcard, Descriptive, Custom HTML, Program, Try Yourself

Section 2: LaTeX & Custom HTML Prompt Engineering Rules

- The 11-Point LaTeX Math Guide & Escaping Rules (latexGuide)
- Custom HTML Glassmorphism Design System & DuoMessageChannel Contract

Section 3: Complete AI Prompt Inventory & Generation Pipeline

- Multi-stage Batched TOC Pipeline (chapterList, sectionList, unitManifest, generateLessonsPrompt)
- Handout, Syllabus, Knowledge-Only, Density, and Metacognitive Prompts

1. Interactive Slide Types (Part 1: Foundational & Practice)

Duofy supports 14 distinct interactive slide types to enforce active recall and practice-first learning.

THEORY

Direct Fact & Concept Presentation

- Direct, factual theory presentation (max ~70 words / 4 sentences).
- Plain definitions without storytelling or narrative padding.
- Full Markdown & LaTeX inline/display math support.

```
JSON: {"type": "theory", "content": "Newton's Second Law..."}
```

CONCEPT PIECES

Bite-Sized Bullet Flow

- Complex concepts split into short sequential statements ().
- Prevents wall-of-text fatigue with progressive disclosures.
- Ideal for multi-stage definitions or core principles.

```
JSON: {"type": "concept_pieces", "content": "Line 1Line 2"}
```

QUIZ

Multiple Choice Question

- Exactly 4 textually distinct options (1-5 words max per option).
- Exactly 1 option marked with `isCorrect: true`.
- Every option contains a 1-line pedagogical explanation.

```
JSON: {"type": "quiz", "content": "What is unit of force?", "options": [{"text": "Newton", "isCorrect": true, ...}]}
```

FILL IN BLANK

Interactive Word Placement

- Question content contains `\_\_\_` blanks (prefer single blank).
- `blankAnswer`: exact comma-separated correct token(s).
- `blankDistractors`: 3 wrong words for user chip selection.

```
JSON: {"type": "fill_in_blank", "content": "Force is ___.", "blankAnswer": "vector", "blankDistractors": ["scalar"...]}
```

ONE WORD

Typed Free Recall

- Direct question requiring user to type single exact answer word.
- No options or distractors shown — tests strict memory retrieval.
- Case-insensitive & whitespace-trimmed evaluation.

```
JSON: {"type": "one_word", "content": "SI unit of power?", "blankAnswer": "watt"}
```

NUMERICAL

Quantitative Calculation

- Quantitative problem requiring a calculated number input.
- `numericAnswer`: target double/int value.
- `numericTolerance`: acceptable margin of error (e.g., 0.01).

```
JSON: {"type": "numerical", "content": "Compute 5*9.8", "numericAnswer": 49.0, "numericTolerance": 0.1}
```

PROOF / STEP_BY_STEP

Interactive Derivation & Multi-Step Logic

- Maps multi-stage mathematical derivations, physics proofs, or logical procedures into discrete steps.
- Each step in `interactiveSteps` can be static (`stepText`) or interactive (`prompt` + `options`).
- Optional `canvasPrompt` renders visual geometric figures or free-body diagrams alongside logic.

```
JSON: {"type": "proof", "content": "Prove Pythagorean Theorem", "interactiveSteps": [{"stepText": "Given a right triangle..."}, {"prompt": "What is c^2?", "options": [{"text": "a^2 + b^2", "isCorrect": true}]}]}
```

PEDAGOGICAL FUNCTION OF TYPES 1-7

Phase A: Encoding (Theory & Concept Pieces)

presents raw factual knowledge directly without storytelling fluff, building structured mental models.

Phase B: Discrimination & Recognition (Quiz & Fill in Blank)

forces the learner to discriminate correct terms from plausible distractors with detailed explanations.

Phase C: Production & Computation (One Word, Numerical, Proof)

requires unprompted free recall and multi-step computational/logical execution for deep mastery.

1. Interactive Slide Types (Part 2: Advanced & Applied)

Advanced slide types for sequencing, flashcards, AI essay grading, interactive HTML, and live code runners.

MATCHINGPair Association Matching

- 3-6 ``left`: "...", "right": "...`` objects.
- Used for term↔definition, symbol↔meaning, quantity↔unit.
- Interactive column selection & matching feedback.

JSON: `{ "type": "matching", "content": "Match units.", "matchPairs": [ { "left": "Force", "right": "Newton" } ] }`

ORDERINGSequence & Algorithm Reorder

- ``orderItems``: 3-6 items listed in exact correct sequence.
- App shuffles items; user drags into correct chronological order.
- Used for procedures, derivations, and historical events.

JSON: `{ "type": "ordering", "content": "Sequence steps.", "orderItems": [ "Step 1", "Step 2", "Step 3" ] }`

FLASHCARDRecall-Flip Spaced Review

- Front side: recall prompt (``content``); Back side: answer (``blankAnswer``).
- Self-graded ("Got it right" / "Need review").
- Misses automatically enter spaced repetition review queue.

JSON: `{ "type": "flashcard", "content": "State Ohm's Law", "blankAnswer": "V = I * R" }`

DESCRIPTIVEParagraph & Photo Essay

- Open-ended essay or calculation requiring explanation.
- Supports text box typing + optional photo upload of handwritten work.
- Evaluated by AI tutor with rubric feedback & score.

JSON: `{ "type": "descriptive", "content": "Explain photosynthesis", "title": "Essay Task" }`

CUSTOM_HTMLInteractive Canvas & Web Simulators

- Embedded transparent WebView running full HTML5/CSS3/JavaScript code.
- Used for interactive circuit simulators, vector sliders, sorting visualizers, and physics games.
- JavaScript MUST trigger ``DuoMessageChannel.postMessage("complete")`` when solved!

JSON: `{ "type": "custom_html", "content": "Simulate vector addition", "interactiveCanvasHtml": "<div style='color:white'>...<script>DuoMessageChannel.postMessage('complete');</script></div>" }`

PROGRAMCode Fill-in-the-Blank

- ``code``: code snippet containing exactly one ``__`` blank.
- ``blankAnswer``: correct token (e.g. ``print``).
- ``blankDistractors``: 3 code tokens for chip selection.

JSON: `{ "type": "program", "language": "python", "code": "def run(): __('hi')", "blankAnswer": "print" }`

TRY YOURSELFLive Code Runner Sandbox

- Practice-first hands-on code runner loaded with starter code.
- Executes live Python (SeriousPython + NumPy / Matplotlib / Sklearn).
- Captures stdout, stderr, and plots SVG/base64 graphics.

JSON: `{ "type": "try_yourself", "language": "python", "code": "import numpy as np...", "packages": [ "numpy" ] }`

CLASSIFICATION MATRIX OF ALL 14 SLIDE TYPES

Category 1: Factual & Concept Exposition	theory, concept_pieces	Direct definition & bullet flow
Category 2: Active Recall & Recognition	quiz, fill_in_blank, one_word	Distractor discrimination & recall
Category 3: Quantitative & Logical Derivations	numerical, proof, step_by_step	Math calculation & logic steps
Category 4: Structure, Sequence & Spaced Review	matching, ordering, flashcard	Pair matching, sequence & memory queue
Category 5: Applied Practice & Simulators	descriptive, custom_html, program, try_yourself	Essay, canvas webview & Python IDE

2. Prompt Engineering: LaTeX Math Rules (11-Point Guide)

Inlined into every lesson JSON prompt to prevent backslash stripping, delimiter errors, and JSON malformation.

1. JSON DOUBLE ESCAPING MANDATE (CRITICAL RULE)

Every backslash in LaTeX output MUST be written as double-backslash inside JSON strings.

Correct: `\frac{1}{2}`, `\int_0^1`, `\sqrt{x}`, `\alpha` | INCORRECT: `\frac{1}{2}` (causes silent JSON syntax error or backslash loss).

Correct JSON String: `"The integral is \int_0^1 x^2 \, dx = \frac{1}{3}\$"`

2. Inline vs Display Math Delimiters

- Inline math (sentence flow): wrap in ``$ ... $``
Example: ``The area is $\pi r^2$.``
- Display math (centered equation): wrap in `$$ ... $$`
Example: `$$\int_a^b f(x)dx = F(b) - F(a)$$`
- NEVER mix delimiters (`$$$ x $``) or nest delimiters.
- NEVER put short single equations on their own line.

3. LaTeX Command Escaping

- Fractions: `\frac{a}{b}` (braced numerator & denominator)
- Multi-char Exponents/Indices: `x^{10}`, `a_{ij}` (MUST brace)
- Greek Letters: `\alpha`, `\beta`, `\theta`, `\pi`, `\Sigma`
- Operators: `\int`, `\sum`, `\lim_{x \to 0}`, `\sin`, `\log`
- Multiplication: use `\cdot` or `\times` (NEVER Markdown `**`)
- Comparisons: `\leq`, `\geq`, `\neq`, `\approx`

4. Aligned Equations & Four-Backslash Linebreaks (`\\``)

Multi-line aligned equations MUST wrap in `$$ ... $$` and use `\begin{aligned} ... \end{aligned}` with `&`` for alignment columns.
Linebreaks in JSON-encoded aligned equations require FOUR backslashes (`\\`` = 2 for LaTeX `\\`` doubled for JSON string).

```
$$\begin{aligned}
x &= a + b \\
&= c \end{aligned}$$
```

RULES 5 TO 11: DELIMITER RESTRICTIONS & FAILURE MODE PREVENTION

5. Matrices & Determinants	<code>\$\$\begin{pmatrix} a & b \\ c & d \end{pmatrix}\$\$</code> (bmatrix for square brackets).
6. Chemistry & Units	Units in plain text outside math (<code>5 m/s`</code> ). Chemistry: plain <code>`H₂O`</code> or <code>\text{H}_2\text{O}`</code> .
7. Safe Punctuation	Never use Markdown <code>**</code> in math. Never use <code>`_`</code> except for braced subscripts (<code>`a_{ij}`</code>).
8. Contextual Prose	Slot math inline inside descriptive prose; do not leave floating math without surrounding sentence.
9. Forbidden Syntax	NEVER emit single <code>\frac`</code> , <code>\( ... \)`</code> , <code>\[... \]`</code> , HTML <code><sup>`</code> , or raw unicode math (<code>½`</code> , <code>√`</code> , <code>f`</code>).
10. Plain Numeric Results	For <code>`numerical`</code> slides, <code>`numericAnswer`</code> MUST be a raw JSON number (<code>42.0`</code>), NOT <code>"\$42\$"</code> .
11. No LaTeX in Fill-in-Blank / One-Word	NEVER use <code>`\$`</code> or <code>\$\$`</code> in <code>`fill_in_blank`</code> or <code>`one_word`</code> questions, answers, or distractors!

WHY RULE 11 IS CRITICAL FOR USER EXPERIENCE:

In ``fill_in_blank`` and ``one_word`` slides, the user must select or type answer chips directly. If the LLM wraps answers in LaTeX delimiters (e.g. ``$vector$``), the user would be forced to type literal ``$`` characters or chip buttons would display raw LaTeX syntax. Plain text formatting ensures seamless interactive chip & keyboard input.

2. Custom HTML Canvas Design System & Web View Protocol

Glassmorphism theme palette, responsive layout requirements, and the DuoMessageChannel bridge protocol.

CUSTOM HTML APP THEME PALETTE & GLASSMORPHISM RULES

Primary Blue
#1cb0f6

Success Green
#58cc02

Warning Orange
#ff9600

Error Red
#ff4b4b

Accent Violet
#ce82ff

- App Background: `#0b0f19` | Surface Background: `rgba(255, 255, 255, 0.05)` with `1px solid rgba(255, 255, 255, 0.1)` border.
- Responsiveness: Containers MUST use Flexbox / CSS Grid / percentages / vh / vw. NEVER use hardcoded pixel widths/heights.
- DuoMessageChannel Protocol: The webview page MUST call `DuoMessageChannel.postMessage("complete")` when solved!

DUOMESSAGECHANNEL JAVASCRIPT <-> FLUTTER WEBVIEW PROTOCOL



FEW-SHOT CUSTOM HTML CODE EXAMPLES IN PROMPT SERVICE

Example A: Interactive Circuit Sandbox

```
<div style="color:white; display:flex; flex-direction:column; align-items:center;">
<button id="switchBtn" style="background:#1cb0f6;">Toggle Switch</button>
<svg><circle id="bulb" fill="#555"/></svg>
<script>on = !on; if(on) { DuoMessageChannel.postMessage("complete"); }</script></div>
```

Example B: Vector Addition Simulator

```
<div style="color:white;">Vx: <input type="range" id="vx"> Vy: <input type="range" id="vy">
<script>function check() { len = Math.sqrt(vx*vx + vy*vy);
if(Math.abs(len - 5.0) < 0.1) DuoMessageChannel.postMessage("complete"); }</script></div>
```

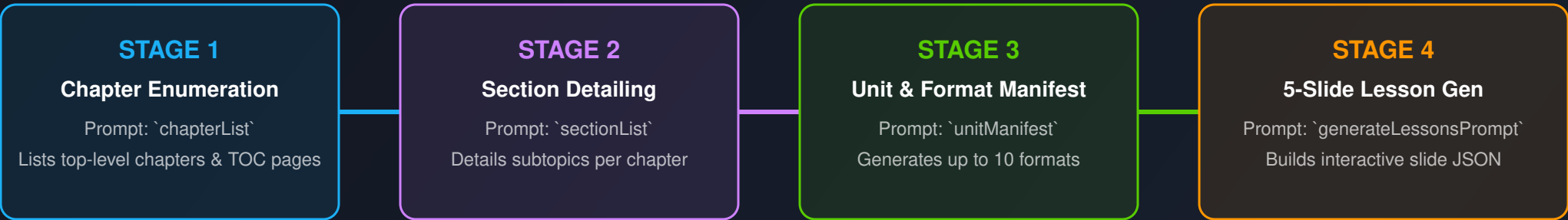
Example C: Interactive Sorting Algorithm Visualizer

```
<div style="color:white;"><div id="bars"></div><button id="step" style="background:#58cc02;">Step Sort</button>
<script>document.getElementById("step").onclick = () => {
if(!swapped) DuoMessageChannel.postMessage("complete"); };</script></div>
```

3. Multi-Stage Batched Course Generation Pipeline

Duofy breaks large textbook/course creation into focused, multi-stage LLM prompts to prevent context drift and missing topics.

BATCHED PROMPT PIPELINE FLOWCHART



Alternative Course Source Pipelines Supported by PromptService:

- 1. Textbook PDF / TOC Flow:** Stage 1 `chapterList` -> Stage 2 `sectionList` (preserves TOC printed page numbers).
- 2. Handout Document Flow:** One-shot `handoutSkeleton` mapping absolute PDF page bounds [1, totalPages].
- 3. Syllabus-Guided Flow:** Stage 1 `syllabusChapterList` -> Stage 2 `syllabusSectionList` matching syllabus modules.
- 4. Knowledge-Only Flow:** Stage 1 `knowledgeChapterList` -> Stage 2 `knowledgeSectionList` (no source PDF).
- 5. Custom Index Text Flow:** `customIndexChapterList` & `parseCustomIndexText` strictly parses user's outline.

GENERATION DENSITY THRESHOLDS (densityBlock)

LOW DENSITY (Easy Review)

- Units per Section: 1 to 2 units | Lessons per Unit: 6 to 8 lessons | Slides per Lesson: EXACTLY 5 slides.
- Ideal for quick refresher courses or rapid surface-level reviews.

MEDIUM DENSITY (Standard - Default)

- Units per Section: 3 to 4 units | Lessons per Unit: 8 to 12 lessons | Slides per Lesson: EXACTLY 5 slides.
- Balanced cognitive load for standard semester learning.

HIGH DENSITY (Deep Dive)

- Units per Section: 5 to 6 units | Lessons per Unit: 12 to 16 lessons | Slides per Lesson: EXACTLY 5 slides.
- Detailed coverage for technical subjects, STEM courses, and exam preparation.

EXTRA & MAX DENSITY (Comprehensive / Exhaustive)

- Extra Density: 7 to 8 units, 16 to 20 lessons per unit.
 - Max Density: 9+ units, 20 to 25 lessons per unit (exhaustive problem breakdown).
- Translates entire textbook exercise sets into individual interactive slide problems one-by-one.

4. Complete AI Prompt Inventory & Service Mapping

Full catalog of all prompt constants, service invocation sites, and AI feedback loops in Duofy.

PROMPT CONSTANT	SOURCE SERVICE	FUNCTION & STAGE	OUTPUT FORMAT
chapterList	PromptService	Stage 1 PDF TOC chapter enumeration	JSON (chapters[])
sectionList	PromptService	Stage 2 subtopic detailing per chapter	JSON (sections[])
generateLessonFormatsPrompt	PromptService	Stage 3 custom lesson format creation (max 10)	JSON (lessonFormats[])
unitManifest	PromptService	Pedagogical unit metadata & Bloom's level	JSON (units[])
generateLessonsPrompt	PromptService	Builds 5 interactive slides per lesson	JSON (lessons[])
handoutSkeleton	PromptService	One-shot course skeleton for PDF handouts	JSON (modules[])
syllabusChapterList	PromptService	Syllabus-aligned chapter enumeration	JSON (chapters[])
knowledgeChapterList	PromptService	Textbook-free course structuring	JSON (chapters[])
customIndexChapterList	PromptService	User custom TOC outline parsing	JSON (chapters[])
slideRulesBlock	PromptService	Single slide regeneration & custom slide rules	JSON (slide)
latexGuide	PromptService	Inlined 11-point LaTeX formatting rules	Text Rule Block
_customHtmlGuide	PromptService	Inlined HTML5 canvas theme & postMessage guide	Text Rule Block
pedagogyBlock	PromptService	Practice-first coding & retrieval practice	Text Rule Block
LessonAssistantChat	LessonAssistantChat Widget	Contextual AI tutor prompt for current slide	Streamed Chat
MetacognitionService	MetacognitionService	Learner model assessment & diagnostic review	JSON Diagnostic

DOCUMENTARY SUMMARY & ARCHITECTURAL STRENGTHS

1. Strict Structural Reliability

By separating TOC parsing into batched, single-purpose API calls (`chapterList` -> `sectionList` -> `unitManifest``), Duofy completely eliminates missing chapters, merged subtopics, and inconsistent TOC structures.

2. Error-Free Math & Code Execution

The 11-point `latexGuide`` prevents backslash stripping and bad math delimiters, while the embedded `SeriousPython`` pure-Python NumPy / Matplotlib / Sklearn stubs guarantee accurate code execution and visualization.

3. Active Retrieval & Practice-First Pedagogy

With 14 specialized slide types spanning practice-first coding runners (`try_yourself``, `program``), custom webview canvas apps (`custom_html``), and interactive recall questions, Duofy ensures active learning and knowledge retention.

4. Dynamic Design System & Web Bridge

The Glassmorphism theme palette and `DuoMessageChannel`` `postMessage`` protocol allow LLMs to author complete interactive physics simulators, circuits, and algorithm games directly within the learning environment.

DUOFY PDF DOCUMENTARY GENERATED SUCCESSFULLY

All 14 slide types, LaTeX guidelines, custom HTML protocols, and AI prompt pipelines documented.